게임콘텐츠2

Section 2-1 : PlayFab

- Account Registration (계정 등록)
- Download and Install (다운로드 및 설치)
- Signup and Login (회원가입 및 로그인)
- Set and Get the player data (플레이어 데이터 설정 및 불러오기)
- Final Result Preview (최종 결과 미리보기)

You will learn how to set and get the player data



Gemini Notebook

PlayFab이란 무엇인가?

> PlayFab은 관리되는 게임 서비스, 실시간 분석 및 LiveOps를 사용하는 라이브 게임을 위한 완벽한 백 엔드 플랫폼입니다.

비용 및 수익 최적화

수익을 높이고 플레이어 참여를 높이는 동시에 비용을 절감하는 데 도움을 줍니다.

개발 진입 장벽 완화

PlayFab의 백 엔드 서비스는 게임과 함께 확장되고 플레이어의 참여, 유지, 수익 창출에 도움이 되는 비용 효율적인 개발 솔루션인 대규모 스튜디오와 소규모 스튜디오를 제공하여 게임 개발자의 진입 장벽을 낮춥니다.

인텔리전트 클라우드 활용

PlayFab을 활용하면, 개발자가 인텔리전트 클라우드를 사용하여 게임을 빌드 및 운영하고, 게임 데이터를 분석하고, 전반적인 게임 환경을 개선할 수 있습니다.

Gemini Notebook

PlayFab 핵심 기능



완벽한 백 엔드 솔루션을 사용하여 대기 시간이 짧은 멀티 플레이어 서*를 대규모로 빌드, 관리, 실행하는 문제를 제거합니다.



여러 형태의 기본 제공 인증을 사용하여 여러 디바이스에서 플레이어를 추적합니다.



플레이어에게 전체 전사 및 번역 서비스로 인게임 채팅을 통해 의사소통할 수 있는 기능을 제공합니다.



풍부한 플레이어 세그먼트를 만들고 A/B 실험을 실행하여 정교한 LiveOps 도구로 플레이어 참여를 강화하세요.



신속하게 순위표를 만들고 콘텐츠 실험을 실행하여 플레이어 참여를 강화합니다.



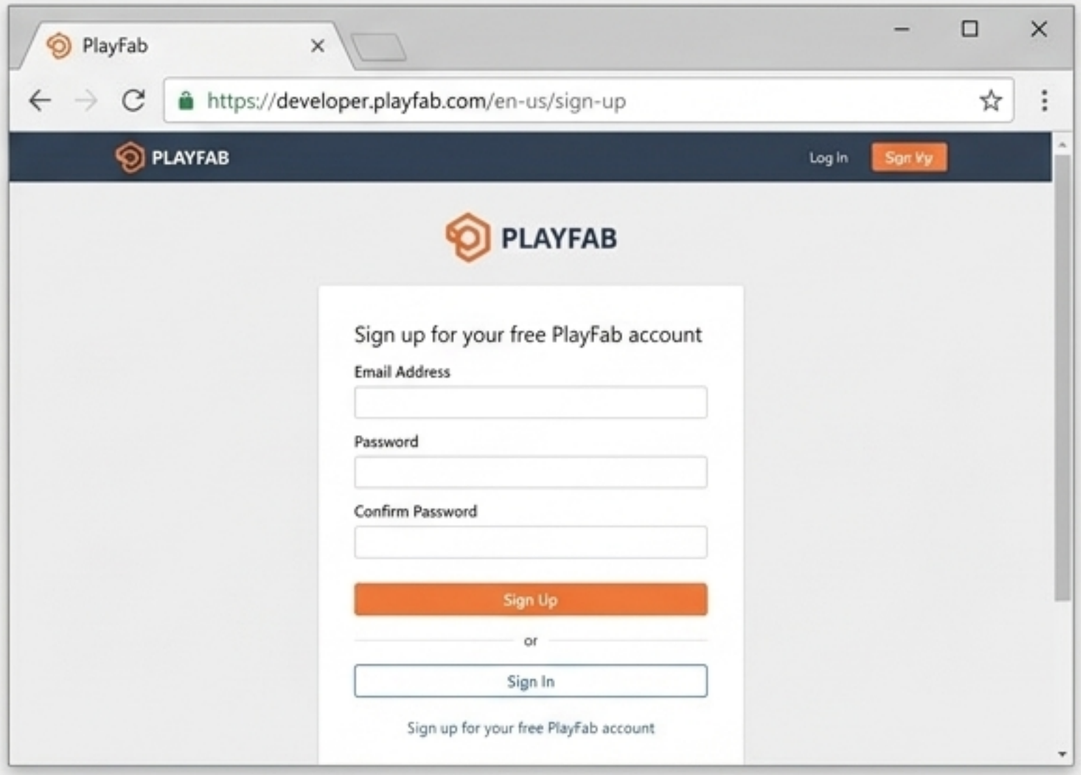
가상 통화를 만들어서 추적하고, 아이템 저장소를 관리하고, 결제를 처리할 수 있는 경제 서비스로 성장을 가속화하세요.

Gemini Notebook

개발 환경 요구 사항 및 계정 등록

요구 사항 (Requirements)

- [x] PlayFab Developer Account (개발자 계정)
- [x] Unity Editor: The PlayFab Unity3D SDK supports Unity Editor version 5.3 (released December 2015) and higher.
- [x] Unity Project (다음 중 하나):
  - A brand new project
  - A guided tutorial project
  - An existing project
- [x] The PlayFab Unity3D SDK



계정 등록: https://developer.playfab.com/en-us/sign-up

Gemini Notebook

PlayFab SDK 다운로드 및 설치

1 **Step 1. SDK 다운로드**

웹사이트 접속 후 SDK 다운로드 (https://learn.microsoft.com/ko-kr/gaming/playfab/sdks/unity3d/)



2 **Step 2. 유니티 프로젝트 열기**

Open your Unity project 후 패키지 임포트 실행 (Import unity package in the Unity Editor)



3 **Step 3. 확장 패널 오픈**

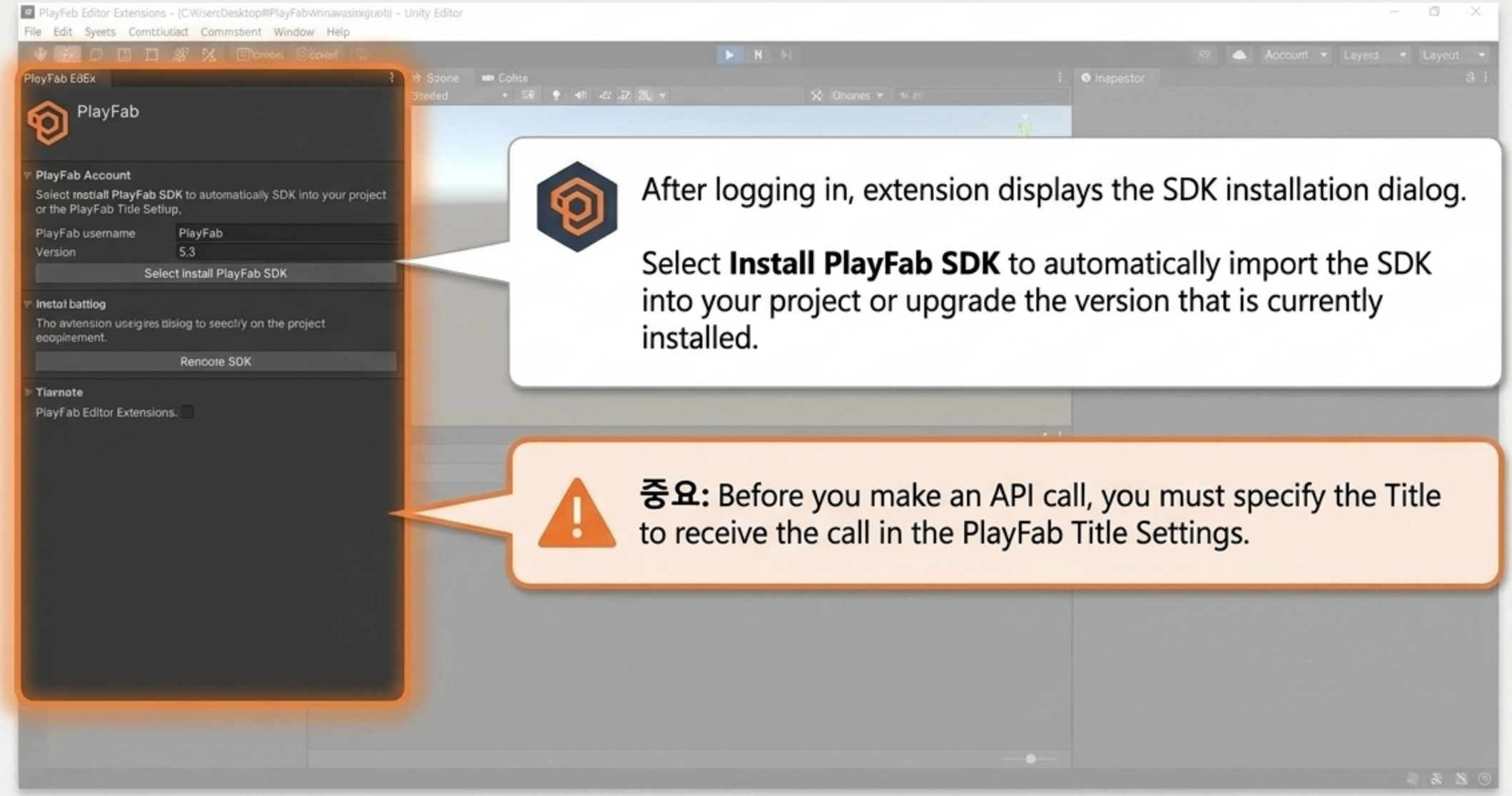
When the import has completed, the PlayFab Unity Editor Extensions panel should open automatically.
(지동으로 열리지 않을 경우 경로: **[Windows – PlayFab – Editor Extensions]**)

4 **Step 4. 로그인**

Log in with your PlayFab username and password.

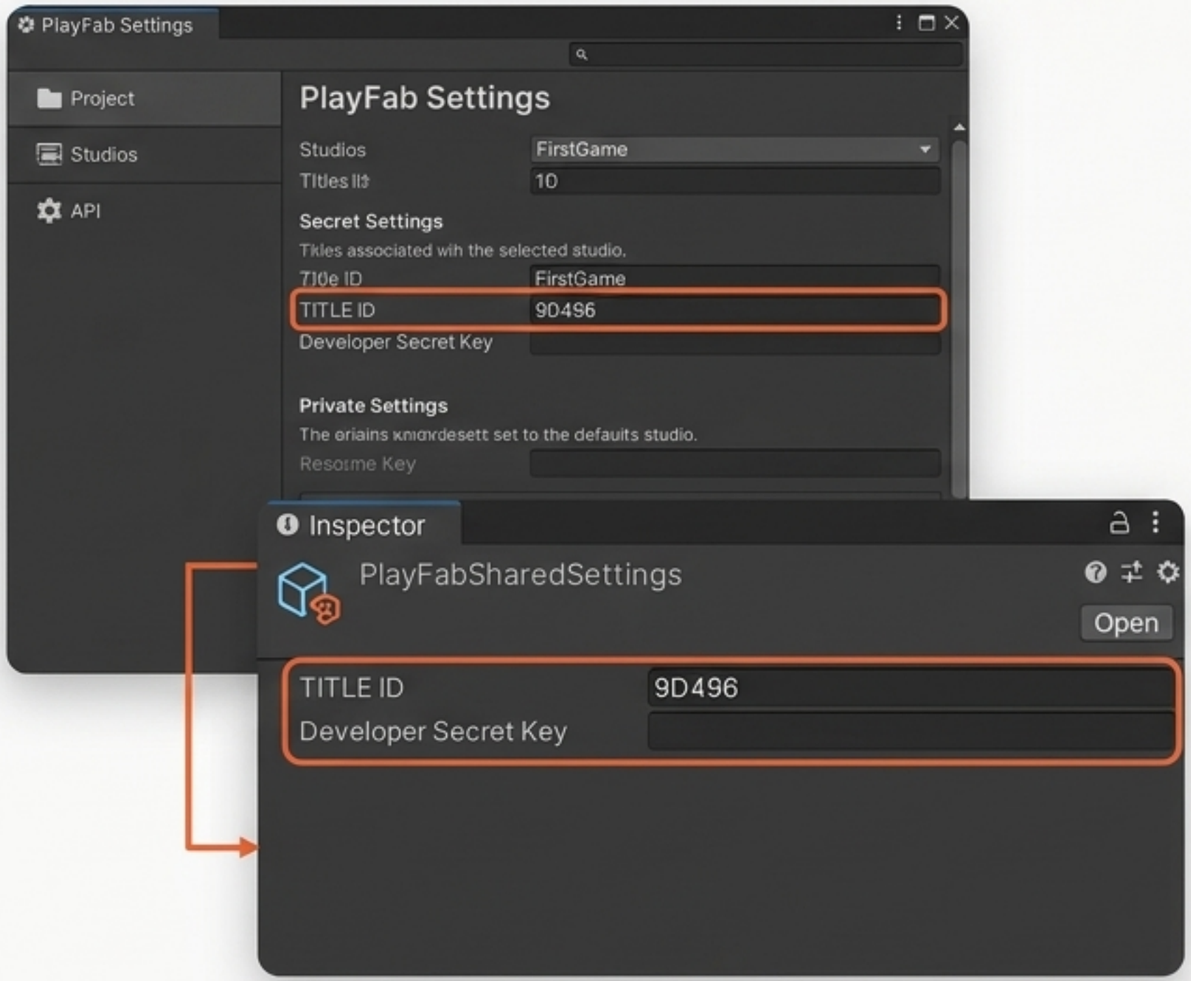
Gemini Notebook

SDK 인스톨 완료 및 초기 설정



Gemini Notebook

PlayFab Shared Settings 구성



메뉴 접근:

`[PlayFab - MakePlayFabSharedSettings]`

설정 파일 위치:

`Assets/PlayFabSDK/Shared/Public/Resources/PlayFabSharedSettings.asset`

세부 설정 내용:

- **Title ID 설정**: Select TitleID entry to enter Titles associated with the selected studio. (본 문서에서는 `Title ID: 9D496`, FirstGame 프로젝트 기준)
- **Developer Secret Key**: The Developer Secret Key is automatically set to the default secret key for the Title.

Gemini Notebook

첫 API 호출 스크립트 작성 및 테스트 (PlayFabFirstCall)

- ❑ Create C# Script, named PlayFabFirstCall, in the Scripts folder.
- ❑ Create Empty Object in the Hierarchy panel, then Drag and drop the C# script to the object.
- ❑ Be sure to save all files and return to the Unity Editor.
- ❑ Press the Play button at the top of the editor.

```
using UnityEngine;
using PlayFab;
using PlayFab.ClientModels;

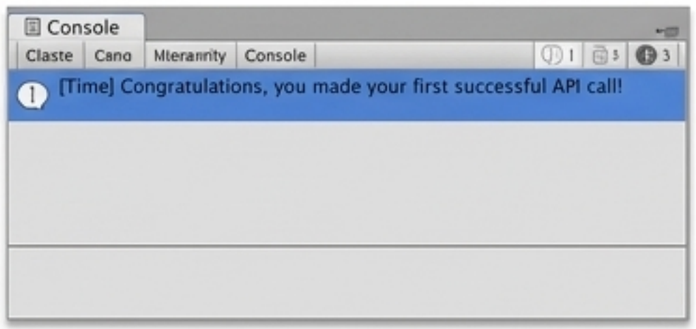
public class PlayFabFirstCall : MonoBehaviour {
    public void Start() {
        if (string.IsNullOrEmpty(PlayFabSettings.staticSettings.TitleId)){
            /* Please change the titleId below to your own titleId from PlayFab Game Manager.
            If you have already set the value in the Editor Extensions, this can be skipped. */
            PlayFabSettings.staticSettings.TitleId = "42";
        }
        var request = new LoginWithCustomIDRequest { CustomId = "GettingStartedGuide", CreateAccount = true};
        PlayFabClientAPI.LoginWithCustomID(request, OnLoginSuccess, OnLoginFailure);
    }

    private void OnLoginSuccess(LoginResult result) {
        Debug.Log("Congratulations, you made your first successful API call!");
    }

    private void OnLoginFailure(PlayFabError error) {
        Debug.LogWarning("Something went wrong with your first API call.  :(");
        Debug.LogError("Here's some debug information:");
        Debug.LogError(error.GenerateErrorReport());
    }
}
```

Important: You must replace "42" with your specific PlayFab Title ID from Game Manager if it's not already configured.

This executes the first API call to log in with a custom ID. Successful calls trigger OnLoginSuccess, failures trigger OnLoginFailure.



Gemini Notebook

회원가입 및 로그인 UI 생성 세팅



사전 준비 사항:
Create a new title on PlayFab

UI 패키지 임포트 지시어

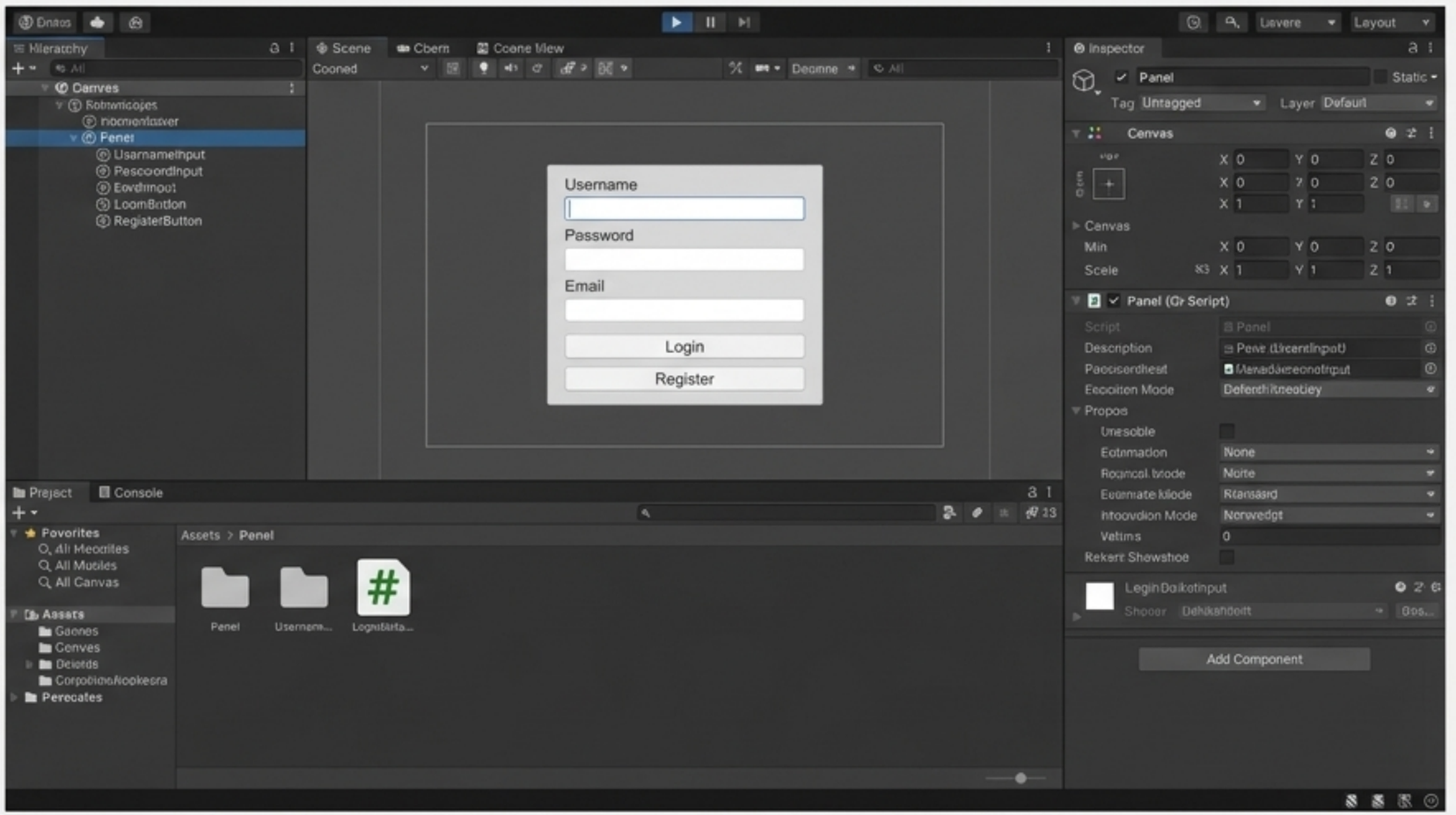
TMPro를 사용할 경우:
TextMeshPro Package Import,
`using TMPro;`

UI – Legacy를 사용할 경우:
`using UnityEngine.UI;`

게임 오브젝트 구성

+

UI 요소를 통제할 **LoginManager** 빈 객체를 생성하고 컴포넌트를 할당할 준비를 합니다.



Gemini Notebook

PlayFabLogin 스크립트 작성 (1/2) - 변수 선언 및 입력값 바인딩



작업 지시어: Create C# Script, named PlayFabLogin, in the Scripts folder. Create Empty Object in the Hierarchy panel (named LoginManager), then Drag and drop the C# script to the object.

PlayFabLogin.cs

```
using UnityEngine;
using PlayFab;
using PlayFab.ClientModels;
using TMPro;

public class PlayFabLogin : MonoBehaviour
{
    public TMP_InputField inputUserID;
    public TMP_InputField inputPassword;
    public TMP_InputField inputEmail;
    public TMP_Text displayMessage;
    private string username;
    private string password;
    private string email;

    // Start is called before the first frame update
    void Start() { PlayFabSettings.TitleId = "9D496";}
    // Update is called once per frame
    void Update() { }
    public void UsernameValueChanged() { username = inputUserID.text.ToString(); }
    public void PasswordValueChanged() { password = inputPassword.text.ToString(); }
    public void EmailValueChanged() { email = inputEmail.text.ToString(); }
}
```

UI 변수 맵핑 선언부

Gemini Notebook

PlayFabLogin 스크립트 작성 (2/2) - API 호출 및 콜백 처리

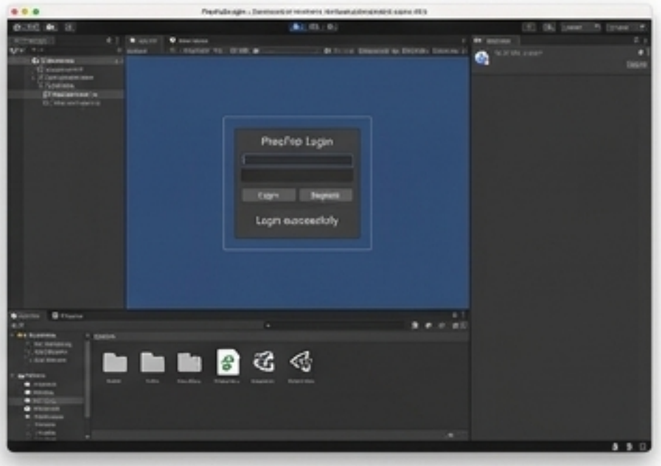
⚠ **유니티 이벤트 연결 필수:** Button의 OnClick(), InputField의 OnValueChanged() 함수를 반드시 설정. 이후 Press the Play button하여 오류 메시지 확인.

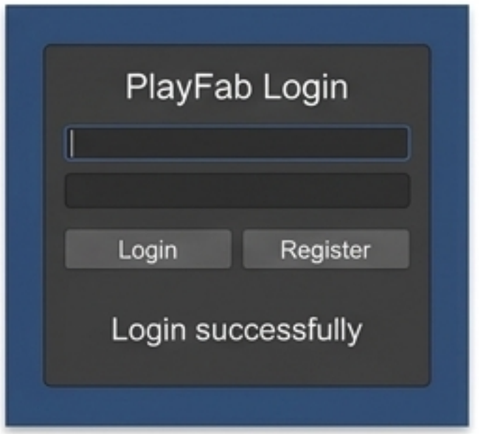
PlayFabLogin.cs

```
public class PlayFabLogin : MonoBehaviour {
    ...
    public void Login() {
        var request = new LoginWithPlayFabRequest { Username = username, Password = password };
        PlayFabClientAPI.LoginWithPlayFab(request, OnLoginSuccess, OnLoginFailure);
    }
    private void OnLoginSuccess(LoginResult result) {
        displayMessage.text = "Login successfully";
        StartGame();
    }
    private void OnLoginFailure(PlayFabError error) {
        Debug.LogWarning(error.GenerateErrorReport());
        displayMessage.text = error.GenerateErrorReport();
    }

    public void Register() {
        var request = new RegisterPlayFabUserRequest { Username = username, Password = password, Email = email };
        PlayFabClientAPI.RegisterPlayFabUser(request, RegisterSuccess, RegisterFailure);
    }
    private void RegisterSuccess(RegisterPlayFabUserResult result) {
        displayMessage.text = "Signup successfully";
    }
    private void RegisterFailure(PlayFabError error) {
        Debug.LogWarning(error.GenerateErrorReport());
        displayMessage.text = error.GenerateErrorReport();
    }

    void StartGame() { Debug.Log("Now, start the game, enjoy it"); }
}
```





Gemini Notebook

플레이어 데이터(Player Data)란 무엇인가?

PlayFab 데이터는 데이터 분석, 스토리지, 처리 및 내보내기를 위한 도구 세트입니다. 타이틀, 플레이어, 캐릭터 및 그룹 데이터 기능과 엔터티, 콘텐츠 배달 네트워크(CDN), 웹후크 같은 데이터 관리/프로비전 기능을 제공합니다.

PlayFab Data

Title Data

게임의 원격 구성을
키-값 쌍으로 관리

Player Data

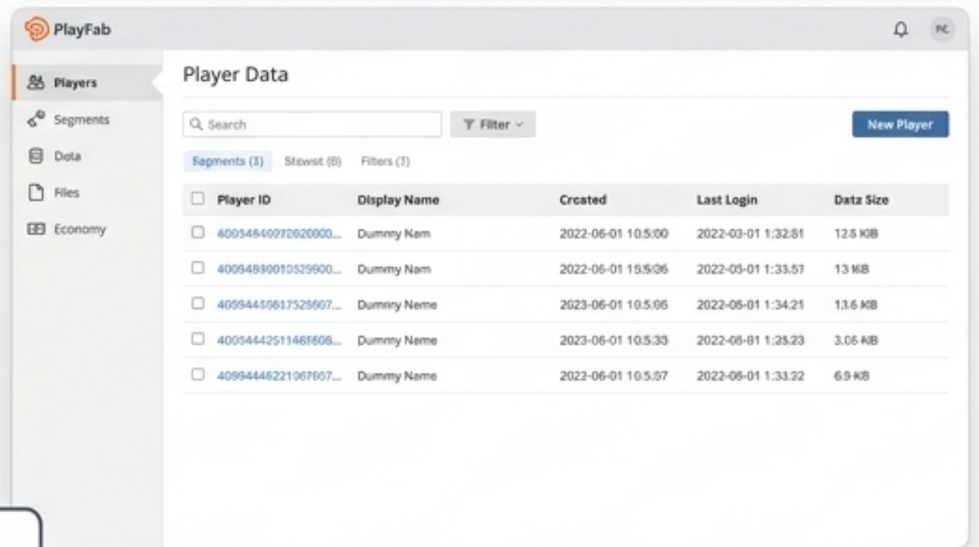
여러 디바이스/게임에서
공유 가능한 플레이어 정보

Entities (권장)

개체의 데이터와 플레이어, 캐릭터,
그룹 전반의 파일을 저장.
(최고의 유연성과 성능을 제공하므로
'모든' 새 타이틀에서 사용 권장)

Player Data / UserData

플레이어의 키/값 쌍 데이터를 저장.
API 함수 이름은 'UserData', 게임
관리자 UI에서는 'Player Data'로 표기
(서로 동일/호환)



Gemini Notebook

플레이어 데이터의 3가지 액세스 모드 비교 (Access Modes)

모드 명칭	읽기/쓰기 권한 (생성 주체)	서버 API	클라이언트 API	특징
Client (클라이언트)	타이틀 클라이언트	-	UpdateUserData (쓰기) / GetUserData (읽기)	타이틀 클라이언트가 자유롭게 읽고 업데이트할 수 있는 데이터.
Read Only (읽기 전용)	서버 전용 쓰기, 클라이언트 열람 가능	UpdateUserReadOnlyData	GetUserReadOnlyData	서버에서 만들거나 업데이트함. 클라이언트는 읽을 수만 있고 업데이트 불가(플레이어 열람 가능, 서버만 수정).
Internal (내부)	서버 전용 (클라이언트 접근 불가)	UpdateUserInternalData	GetUserInternalData	서버에서만 사용할 수 있는 데이터. 서버 전용이며 클라이언트에서는 절대 볼 수 없음.

Gemini Notebook

데이터 저장(Set) 및 불러오기(Get) 스크립트 구현 (Quickstart)

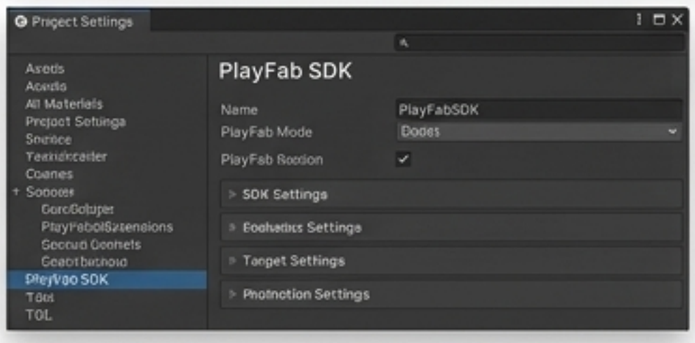
PlayFabLogin 클래스에 메소드 추가. SetUserData는 로그인한 플레이어의 데이터를 생성/업데이트하고, GetUserData는 지정된 플레이어 데이터를 검색. `using System.Collections.Generic;` 선언 필수.

SetUserData

```
void SetUserData() {
    PlayFabClientAPI.UpdateUserData(new UpdateUserDataRequest() {
        Data = new Dictionary<string, string>() {
            {"Ancestor", "Arthur"},
            {"Successor", "Fred"}
        }
    },
    result => Debug.Log("Successfully updated user data"),
    error => {
        Debug.Log("Got error setting user data Ancestor to Arthur");
        Debug.Log(error.GenerateErrorReport());
    });
}
```

GetUserData

```
void GetUserData(string myPlayFabId) {
    PlayFabClientAPI.GetUserData(new GetUserDataRequest() {
        PlayFabId = myPlayFabId,
        Keys = null
    }, result => {
        Debug.Log("Got user data:");
        if (result.Data == null ||
            !result.Data.ContainsKey("Ancestor"))
            Debug.Log("No Ancestor");
        else
            Debug.Log("Ancestor: "+result.Data["Ancestor"].Value);
    }, (error) => {
        Debug.Log("Got error retrieving user data:");
        Debug.Log(error.GenerateErrorReport());
    });
}
```



Gemini Notebook

적용 사례: 슈팅 게임의 데이터 저장 설계 모델

Q. 슈팅 게임에서 어떤 데이터를 저장하면 좋을까? (What kind of data should be stored in a shooting game?)

Database Schema

플레이어 정보
플레이어 이름, 레벨, 경험치, 능력치, 장비 등

게임 진행 상황
현재 플레이 중인 스테이지, 획득한 점수/아이템, 적군의 수, 남은 시간

게임 설정
게임 난이도, 음량, 해상도, 조작 방법 등

게임 기록
이전 클리어한 스테이지, 얻은 업적, 최고 점수

게임 내 상점 정보
상점 판매 아이템 목록/가격, 보유한 돈

로그인 정보
서비스 이용에 필요한 계정 정보

게임 이벤트 정보
이벤트 진행 상황, 보상

게임 서버와의 통신 정보
통신 상태, 서버 연결 정보

결론: 이러한 정보들을 PlayFab에 저장함으로써, 이전의 진행 상황을 온전히 유지하고 게임 서비스 보안성 향상 및 사용자 경험을 획기적으로 개선할 수 있습니다.

Gemini Notebook